Темы занятия

• Определение структуры
• Создание объекта структуры
• Инициация полей по умолчанию
• Конструкторы структуры
• Копирование структуры
• Типы значений и ссылочные типы
• Стек и куча, сборка мусора
• Составные типы и копирование значений
• Ссылочные типы внутри типов значений
• Объекты как параметры методов
• Область видимости переменных и констант
• Класс Program, метод Main, программы верхнего уровня
• Пространства имён

Домашние задания

Постановка

1. Выполните **индивидуально Задание** 1.
2. Выполните **индивидуально Задание** 2.
3. Выполните **совместно** в рабочей группе **Задание** 3.
4. Сформулируйте **список вопросов**, возникших в ходе выполнения **заданий** 1, 2 и 3.
5. **Передайте** список вопросов **Лидеру** группы.
6. (**Выполняет Лидер группы**) **Оформите Отчёт** о проделанной работе. Прикрепите список вопросов от каждого участника рабочей группы к Отчёту. (Требования к отчёту см. ниже.)
7. (**Выполняет Лидер группы**) **Отправьте Отчёт** о проделанной работе и код проекта на указанную на занятии электронную почту.

Задание 1

Задание для разминки. Дополните структуру `Auditorium` (см. `//TODO: ⚠️`):

```
// Аудитория университета (значимый тип)
struct Auditorium
```

```
{
    public string Number;    // номер аудитории
    public string Building;  // корпус
    public int Seats;        // количество мест

    //TODO: ⚠️ Главный конструктор (number, building, seats) ⚠️

    //TODO: ⚠️ Упрощённый конструктор (number) : this(number, "Главный
корпус", 0) ⚠️

    public string Describe() => $"ауд. {Number}, {Building}, мест: {Seats}";
}
```

Затем в методе `Main`:

1. создайте аудиторию через **конструктор с параметрами**;
2. создайте аудиторию через `default` и выведите её описание — объясните в комментарии, почему получились именно такие значения;
3. сделайте **копию** первой аудитории и в копии сменяйте количество мест — убедитесь, что оригинал не изменился;
4. сделайте ту же копию через выражение `with`;
5. напишите метод `Reserve(ref Auditorium auditorium, int seats)`, который уменьшает количество свободных мест, и метод `Show(Auditorium auditorium)`, который печатает описание. Продемонстрируйте разницу.

Задание 2

Индивидуально. Напишите **два коротких примера**:

1. Пример, где две переменные **класса** указывают на один объект: измените объект через одну переменную и покажите, что изменился он и для другой.
2. Пример, где та же операция **со структурой** не изменяет оригинал: покажите вывод до и после.

Дополнительно ответьте **одной строкой** на каждый вопрос:

1. Где хранится объект класса, а где — локальная переменная типа `int`?
2. Когда сборщик мусора может освободить объект?
3. Почему структура с полем ссылочного типа копируется не полностью «в глубину»?

Задание 3 (групповое)

Продолжаем развивать наш проект с **Пользователем**.

Теперь нашей целью будет разработка **Корзины Покупателя**, которую можно наполнить **Товарами** имеющими **Цену** в условных **денежных единицах**, часть которой можно заплатить **бонусными баллами**, а для расчёта финальной цены на товары в корзине мы должны учесть **Персональную Скидку Пользователя** и его **Баланс Бонусных Баллов**.

Для осуществления задуманного нам потребуется реализовать следующие классы и структуры:

1. класс **Товара** (ID, Название, цена за единицу)
2. класс **Товарной Позиции** в Корзине (ID, ID Товара, количество единиц товара в позиции).
3. класс **Корзины** Пользователя (ID, ID Пользователя, список Товарных Позиций)
4. структуру **Рассчётной Цены** корзины (считаем, что все товары могут быть проданы со скидкой)(**Исходная цена**, **Скидка** в виде множителя от 0.01 до 1.00, где 0.01 - скидка 99%, а 1.00 - скидка 0%, **Размер Скидки в Деньгах**)
5. структуру **Состояния Баланса Покупателя** (содержащую количество **бонусов** и количество денежных **единиц**, а так же **долю итоговой цены, которую можно оплатить бонусами** в виде множителя от 0.0 до 0.5, где 0.0 - это 0% и 0.5 - это 50% итоговой цены можно оплатить бонусами).

Кроме того, нужно будет дополнить **Профиль Пользователя**, добавив в него поля:

1. **Баланс в денежных единицах** (с плавающей точкой, не может быть меньше 0.0)
2. **Баланс в бонусах** (с плавающей точкой, не может быть меньше 0.0)
3. **Персональную скидку** (с плавающей точкой, от 0.01 до 1.00, где 0.01 - скидка 99%, а 1.00 - скидка 0%)
4. **Разрешённая доля оплаты бонусами** (с плавающей точкой, от 0.0 до 0.5, где 0.0 - это 0% и 0.5 - это 50% итоговой цены можно оплатить бонусами)
5. **Совершённые Покупки** - Список **Товарных Позиций**, отражающий приобретения Покупателя.

Нужно **реализовать сценарии** с **успешной** и **неуспешной оплатой покупок** покупателем (**ниже - успешный сценарий** с описанием возможных действий на случай неуспешного - п. 12-13):

1. **Пользователь** пополнил **баланс** на 10000 **условных единиц** денег (**УЕ**).
2. **Пользователю** подарили 10% от суммы **бонусами**.
3. **Пользователю** разрешили **оплачивать** до 10% итоговой стоимости Корзины его **бонусами**.
4. **Пользователю** предоставили персональную **скидку** 5%.

5. **Пользователь Заказывает** 3 **Товара** "Тарелка" стоимостью 1000 УЕ, 2 **Товара** "Приборы" стоимостью 2000 УЕ.
6. Заказанные **Товары** попадают в **Корзину Покупателя** в виде **Товарных Позиций**.
7. **Корзина Покупателя** возвращает **Расчётную Цену** с учётом **персональной скидки**.
8. **Пользователю** показывают итоговую **цену товаров в корзине** с учётом Персональной Скидки и **Размер Скидки в Деньгах**, вызвав методы **Расчётной Цены** "покажи итоговую цену" и "покажи размер скидки в деньгах".
9. **Пользователь** пытается **оплатить** итоговую цену **бонусами** на столько, на сколько ему позволяет **Разрешённая доля оплаты бонусами**, остальное **оплачивает** из личного **баланса в УЕ**, передавая **Состояние Баланса Покупателя** и **Расчётную Цену** в **сервисный метод оплаты** (оплата бонусами и деньгами совершается в **одном** вызове метода оплаты!).
10. Если **оплата возможна** (**денег** и **бонусов хватает** на оплату Корзины со скидкой) - **баланс пользователя** приводится **в соответствие с состоянием структуры Состояние Баланса Покупателя** после проведения оплаты, а к списку **Совершённых Покупок** добавляется список **Товарных Позиций из Корзины Покупателя**.
11. **Корзина Покупателя опустошается** - в ней 0 **товаров** с итоговой **ценой** 0 УЕ.
12. Если оплата **не возможна** - **Корзина** остаётся **в исходном состоянии** (Товарные Позиции на месте), **баланс Покупателя** в УЕ и бонусах - остаётся **в исходном состоянии** (списаний не производится). На экране видим **сообщение о нехватке средств** на балансе.
13. После **пополнения** на необходимую сумму **баланса Пользователя** - **пробуем провести успешную оплату**.

Распределение задач по участникам

Участники распределяют между собой реализацию классов и структур.
Лидер - компонует разработанные решения.

Компоновка (задача Лидера)

Лидер рабочей группы:

1. собирает все части в один проект (по одному классу/структуре на файл);
2. пишет точку входа (`Main` или программы верхнего уровня), где:
   - создаёт покупателя и 2-3 заказа;
   - показывает разницу между копированием **ссылки** и **значения**;
   - выводит отчёты об успешных и не успешных операциях оплаты покупок;
   - демонстрирует `with` для Состояния Баланса Покупателя после успешной оплаты покупок;

3. проверяет, что копии не портят оригинальные данные;
4. оформляет отчёт.

Рекомендации по выполнению

1. **Один тип — один файл.** Файл `Money.cs`, `Customer.cs`, `Order.cs`, `OrderService.cs`, `Program.cs`.
2. **Пространства имён:** типы проекта разместите в отдельном пространстве имён, например `Marketplace.Domain`, а сервис — в `Marketplace.Services`. Используйте файловое пространство имён (`namespace Marketplace.Domain;`).
3. **Не делайте изменяемых структур без необходимости.** Если структура может быть `readonly` — сделайте её `readonly`.
4. **Проверяйте данные в конструкторах:** отрицательная сумма, пустое имя, `null` вместо клиента — это ошибки, а не «потом разберёмся».
5. **Обсуждайте общие решения:** названия типов и методов должны быть одинаковыми во всех частях, чтобы компоновка не превратилась в переделку.
6. **Каждый участник** пишет либо небольшой класс, либо как минимум один серьёзный публичный метод сервиса с 1-2 приватными методами внутри.

Требования к отчёту

Отчёт оформляет **Лидер группы**, файл — в формате Markdown: **`readme.md`**.

1. **Титульная часть (в начале документа):** номер занятия, тема, состав рабочей группы (3-4 человека, лидер отмечен), вариант проекта.
2. **Текст исходного задания** — полностью, как он приведён в этом файле.
3. **Для каждого (под)задания:** краткое описание выполненного решения (2-5 предложений о том, что и как сделано, какие решения приняты).
4. **Под каждым описанием — листинг решения (или упоминание конкретного файла класса из числа приложенных к отчёту файлов проекта)** (код с комментариями).
5. **Список вопросов** от каждого участника группы (опционально).
6. **Выводы:** что оказалось сложным, чем структуры отличаются от классов на практике, где в проекте использовано `with`.
7. К отчёту должен быть приложен код проекта: файлы классов или архив со всей структурой проекта.

Критерии оценки группы:

- проект собирается и запускается без правок;
- видна разница поведения структур и классов (копии не портят оригиналы, ссылки — портят);
- соблюдено распределение ролей: каждый участник написал свою часть;
- оформление отчёта соответствует требованиям выше.

Срок сдачи — до вечера пятницы.

Куда присылать выполненные задания

Электронная почта: moysklad-students-homework@mail.ru

В теме письма нужно указать:
ООП 2026 Домашнее задание + номер задания + номе учебной группы + ФИО

номер задания - для текущего занятия - 2.
ФИО - Фамилия Имя О.

Материалы

Структуры

Структура (`struct`) — это **значимый тип**, который, как и класс, объединяет **поля** и **методы**.

Отличия, которые нужно запомнить сразу:

- структура **копируется целиком** при присваивании и передаче в метод;
- структура **не может наследоваться** от класса и не может быть базой для другой структуры;
- у структуры **нет** ссылки на объект в куче — сама переменная **и есть** значение.

Структура хорошо подходит для **небольших** наборов данных без собственной «жизни»: оценки, координаты, денежная сумма, номер аудитории.

Примеры из жизни:
`Point` / `Vector2`
`DateTime` и `TimeSpan`
`Span<T>` и `ReadOnlySpan<T>`

Вопрос для закрепления: чем студенческий билет отличается от студента — почему билет удобно сделать структурой, а студента классом?

Определение структуры и создание объекта структуры

```
// Студенческий билет (значимый тип: это просто набор характеристик)
struct StudentCard
{
    // Поля структуры
    public string? Number;  // номер билета (null у структуры по умолчанию)
    public int Course;      // курс обучения
    public bool IsValid;    // действителен ли билет

    // Конструктор структуры обязан присвоить значения ВСЕМ полям
    public StudentCard(string number, int course)
    {
        Number = number;
        Course = course;
        IsValid = true;
    }

    // Метод структуры
    public string Describe() =>
        $"билет {Number}, {Course} курс, " + (IsValid ? "действителен" :
"недействителен");
}
```

Объект структуры создаётся так же, как объект класса:

```
StudentCard card = new StudentCard("Б-2417", 2);
Console.WriteLine(card.Describe());
```

Вопрос для закрепления: чем описание структуры отличается от описания класса, которое мы делали на прошлом занятии?

Инициация полей по умолчанию

Если конструктор не вызывается явно, структура получает **все поля со значениями по умолчанию** своего типа: `0`, `false`, `null`, `default` для вложенных структур.

```
StudentCard empty = new StudentCard();   // все поля обнулены
StudentCard alsoEmpty = default;         // то же самое
```

```
// Number == null, Course == 0, IsValid == false
Console.WriteLine(empty.Describe());
```

Для класса такого «бесплатного» состояния с корректными значениями нет: там поля инициализируются либо в конструкторе, либо значениями по умолчанию, но объект всё равно нужно создать через `new`.

Вопрос для закрепления: какие значения получат поля структуры `StudentCard`, созданной выражением `default`?

Конструкторы структуры

В структуре можно объявлять конструкторы **с параметрами**. В C# 10+ разрешён и **конструктор без параметров**, но тогда он обязан инициализировать все поля.

```
struct Auditorium
{
    public string Number;
    public int Seats;

    // Конструктор без параметров (C# 10+)
    public Auditorium()
    {
        Number = "не назначена";
        Seats = 0;
    }

    // Конструктор с параметрами
    public Auditorium(string number, int seats)
    {
        Number = number;
        Seats = seats;
    }
}
```

Цепочка конструкторов через `: this(...)` работает так же, как у классов.

Вопрос для закрепления: почему компилятор не позволит конструктору структуры закончиться, не присвоив значения какому-либо полю?

Копирование структуры

При присваивании **копируются все поля**. Дальше оригинал и копия живут независимо.

```
StudentCard original = new StudentCard("Б-2417", 2);
StudentCard copy = original;          // копия всех полей
copy.Course = 3;                      // меняем только копию

Console.WriteLine(original.Course);   // 2 — оригинал не изменился
Console.WriteLine(copy.Course);       // 3
```

С C# 10 для структуры доступно выражение `with`, которое создаёт копию с изменёнными полями:

```
StudentCard moved = original with { Course = 3, IsValid = false };
```

Вопрос для закрепления: изменится ли `original`, если после `StudentCard copy = original;` выполнить `copy.Number = "Б-0000";`?

Типы значений и ссылочные типы

	Тип значения (`struct`, `int`, `bool`, `DateTime`)	**Ссылочный тип (`class`, `string`, массивы)**
Что хранит переменная	само значение	**ссылку** на объект
Где живёт	стек (для локальных переменных) или внутри другого значения	управляемая куча
Копирование	копируется значение	копируется ссылка
Значение по умолчанию	`0`, `false`, нулевая структура	`null`
Наследование	нет	есть

```
// Ссылочный тип: копируется ссылка на один и тот же объект
Student student = new Student("Иван", 19);
Student same = student;
same.SetAge(20);

Console.WriteLine(student.GetAge());  // 20 — это тот же самый объект
```

Хранение типов в памяти: стек и куча, сборка мусора

- **Стек** — быстрая область памяти, где лежат локальные переменные и параметры методов. Значение исчезает автоматически, когда метод завершается.
- **Куча** — область, где лежат объекты классов. Объект живёт, пока на него есть хотя бы одна ссылка.
- **Сборщик мусора (GC)** освобождает объекты в куче, на которые больше никто не ссылается. Работает поколениями (0, 1, 2), вручную вызывать его в обычном коде не нужно.

```
static void Demo()
{
    StudentCard card = new StudentCard("Б-2417", 2);   // значение в стеке
    Student student = new Student("Иван", 19);          // объект в куче,
ссылка в стеке
}
// После выхода из метода card исчезает вместе со стеком,
// а объект student станет «мусором» и будет удалён сборщиком.
```

Вопрос для закрепления: что случится с объектом в куче, если единственная ссылка на него вышла из области видимости?

Составные типы

Составной тип — это тип, который содержит другие типы:

```
// Структура внутри структуры
struct ExamRecord
{
    public string Subject;      // ссылочный тип внутри значения
    public int Mark;            // значение
    public DateTime Date;       // значение (структура)
}

// Значение внутри класса
class StudentProfile
{
    public string Name;
    public StudentCard Card;    // структура внутри ссылочного типа
}
```

Вопрос для закрепления: к какому виду относится `ExamRecord` — к типам значений или ссылочным?

Копирование значений

Правило простое: **копируется то, что лежит в переменной**.

- у типа значения в переменной лежит **значение** → копируется значение;
- у ссылочного типа в переменной лежит **ссылка** → копируется ссылка.

```
StudentCard cardA = new StudentCard("Б-2417", 2);
StudentCard cardB = cardA;      // полная копия

Student studentA = new Student("Анна", 20);
Student studentB = studentA;    // копия ссылки
```

Вопрос для закрепления: сколько объектов в куче появится после выполнения двух строк выше?

Ссылочные типы внутри типов значений

Структура может содержать ссылку. Копирование структуры копирует **ссылку**, а не объект за ней.

```
struct DormPlace
{
    public string Room;
    public Student? Occupant;   // ссылка на объект студента
}

var place = new DormPlace { Room = "412", Occupant = new Student("Иван", 19)
};
var placeCopy = place;          // Room скопирован, Occupant — та же ссылка

placeCopy.Room = "501";         // оригинал не изменился
placeCopy.Occupant!.SetAge(20); // изменился ОБЩИЙ студент
```

Вопрос для закрепления: почему изменение `Room` в копии не видно в оригинале, а изменение `Occupant` — видно?

Объекты как параметры методов

В метод объект класса передаётся **по ссылке**, структура — **по значению**.

```
// Изменяет исходный объект: передана ссылка
private static void RenameStudent(Student student)
{
    student.SetName("Пётр");
}
```

```
// Изменяет только локальную копию: передано значение
private static void ReissueCard(StudentCard card)
{
    card.Number = "Б-0000";
}
```

Если значение структуры нужно изменить в методе, используют `ref`. Если структуру нужно передать **без копирования, но только для чтения** — `in`.

```
private static void MoveToCourse(ref StudentCard card, int course)
{
    card.Course = course;
}
```

Вопрос для закрепления: как передать структуру в метод, чтобы изменения сохранились в вызывающем коде?

Область видимости переменных и констант

Область видимости — часть кода, в которой имя доступно.

- **Локальная переменная** видна от объявления до конца блока `{ ... }`, в котором объявлена.
- **Поле класса** видно во всех методах этого класса.
- **Константа** (`const`) видна в пределах своей области видимости и должна быть известна на этапе компиляции.

```
class Journal
{
    private const int MaxMark = 5;   // константа уровня класса

    public void Print()
    {
        int count = 0;               // видна до конца метода

        if (count == 0)
        {
            int local = 42;          // видна только внутри if
            Console.WriteLine(local + MaxMark);
        }
```

```
        // Console.WriteLine(local); // ошибка: local здесь уже не
существует
    }
}
```

Одноимённые переменные во вложенном блоке **затеняют** внешние — это разрешено, но легко запутаться, поэтому лучше не делать.

Вопрос для закрепления: видна ли переменная, объявленная внутри блока `if`, после закрывающей скобки этого блока?

Пространства имён

Пространство имён — это «папка» для типов, которая позволяет:

- группировать родственные типы;
- избегать конфликтов имён (`University.Student` и `Dormitory.Student`);
- управлять видимостью типов.

```
namespace University.Journal
{
    class Mark
    {
        public int Value;
    }
}
```

Класс Program и метод Main

Классическая точка входа — **статический метод `Main`** в классе `Program`.

```
namespace University;

class Program
{
    static void Main()
    {
        Console.WriteLine("Расписание на день:");
    }
}
```

Программы верхнего уровня

С C# 9 можно писать код **без класса `Program` и без `Main`**: компилятор создаст их сам.

```
// Program.cs — весь файл является телом метода Main
Console.WriteLine("Расписание на день:");
```

Подключение пространства имён

```
using University.Journal;          // подключаем типы пространства имён
using Dorm = University.Dormitory; // псевдоним

var mark = new Mark { Value = 5 };
```

Вложенные пространства имён

Пространства имён можно вкладывать друг в друга — как каталоги.

```
namespace University
{
    namespace Dormitory
    {
        class Room { }
    }
}
```

Пространства имён уровня файла

С C# 10 пространство имён можно объявить **одной строкой на весь файл**:

```
namespace University.Dormitory;   // действует для всего файла

class Room { }
```

Глобальное пространство имён

Типы без объявленного пространства имён попадают в **глобальное** пространство имён — они видны везде, но с ними легко устроить конфликт.

```
class GlobalHelper   // глобальное пространство имён
{
```

```
}
```

Подключение пространства имён по умолчанию

Начиная с .NET 6 в проекте включены **неявные** `using` (`ImplicitUsings`): `System`, `System.Collections.Generic`, `System.Linq` и ещё несколько подключаются автоматически. Кроме этого, можно объявить собственные глобальные `using`:

```
// GlobalUsings.cs
global using System.Text;   // действует во всех файлах проекта
```

Вопрос для закрепления: чем объявление `namespace University.Dormitory;` отличается от `namespace University.Dormitory { ... }`?

Листинги с занятия

Program.cs

```
using MySpace;
class Program
{
    private static readonly string DELIMITER = "\n============\n";

    // Главный метод класса приложения (входная точка)
    static void Main()
    {
        StructDemo();
        StructConstructorDemo();
        StructCopyDemo();

        TypesDemo();

        CompositeDemo();

        MySubSpace.AnyStruct anyStruct = new MySubSpace.AnyStruct();
    }


    /*
    - структура копируется целиком при присваивании и передаче в метод;
    - структура не может наследоваться от класса и не может быть базой для
другой структуры;
```

```
    - у структуры нет ссылки на объект в куче — сама переменная и есть
значение.
    */
    private static void StructDemo(){
        Topic("Структуры C#");

        StudentCard card = new StudentCard("Б-2417", 2);
        Console.WriteLine(card.Describe());
        Console.WriteLine(card);

        // Если конструктор не вызывается явно,
        // структура получает все поля со значениями по умолчанию
        // для их типа: 0, false, null, default для вложенных структур.
        StudentCard empty = new StudentCard();   // все поля обнулены
        StudentCard alsoEmpty = default;         // то же самое

        // Number == null, Course == 0, IsValid == false
        Console.WriteLine(empty.Describe());
        Console.WriteLine(alsoEmpty.Describe());
    }

    private static void StructConstructorDemo()
    {
        Topic("Конструкторы структур");

        Auditorium auditorium = new Auditorium();
        Console.WriteLine(auditorium.Describe());

        // Переиспользуем место в памяти под новую запись
        auditorium = new Auditorium("42",30);
        Console.WriteLine(auditorium.Describe());

        // Все поля со значением по умолчанию (конструктор не вызывается)
        auditorium = default;
        Console.WriteLine(auditorium.Describe());

        auditorium = new Auditorium("A 42");
        Console.WriteLine(auditorium);
    }

    // При присваивании копируются все поля.
    // Дальше оригинал и копия живут независимо.
    private static void StructCopyDemo()
    {
        Topic("Копирование структур");
```

```
        StudentCard original = new StudentCard("Б-2417", 2);
        StudentCard copy = original;          // копия всех полей
        copy.Course = 3;                      // меняем только копию

        Console.WriteLine($"Курс оригинала {original.Course}");   // 2 —
оригинал не изменился
        Console.WriteLine($"Курс копии {copy.Course}");           // 3

        // С C# 10 для структуры доступно выражение `with`,
        // которое создаёт копию с изменёнными полями
        StudentCard moved = original with { Course = 3, IsValid = false };

        Console.WriteLine(original.Describe());
        Console.WriteLine(moved.Describe());
    }

    private static void TypesDemo()
    {
        Topic("Типы данных");

        // Ссылочный тип: копируется ссылка на один и тот же объект
        Student student = new Student("Иван", 19);
        Student same = student;

        Console.WriteLine($"Состояние Student в памяти: {student} и
{same}");

        // Меняем параметр объекта, обратившись к нему через копию ссылки на
него
        same.SetAge(20);
        Console.WriteLine($"Состояние Student в памяти: {student} и
{same}");

        // Возраст 20 — это тот же самый объект, к которому обращаемся по
исходной ссылке
        Console.WriteLine(student.GetAge());

        int a = 1;
        int b = a;
        Console.WriteLine($"Состояние int в памяти: {a} и {b}");

        a = 2;
        Console.WriteLine($"Состояние int в памяти: {a} и {b}");

        Auditorium a1 = new Auditorium("A1",20);
        Auditorium a2 = a1;
```

```
        Console.WriteLine($"Состояние Auditorium в памяти: {a1} и {a2}");
        a1.Seats = 30;
        Console.WriteLine($"Состояние Auditorium в памяти: {a1} и {a2}");
    }

    private static void CompositeDemo()
    {
        PresenceRecord presenceRecord = new(
            new StudentCard("A-1", 2),
            new Auditorium("B-4")
        );
        Console.WriteLine(presenceRecord);
    }

    private static void Topic(string topic){
        Console.WriteLine($"{DELIMITER}{topic}{DELIMITER}");
    }
}
```

StudentCard.cs

```
namespace MySpace;
// Студенческий билет (значимый тип: это просто набор характеристик)
struct StudentCard
{
    // Поля структуры
    public string? Number;  // номер билета (null у структуры по умолчанию)
    public int Course;      // курс обучения
    public bool IsValid;    // действителен ли билет

    // Конструктор структуры обязан присвоить значения ВСЕМ полям
    public StudentCard(string number, int course)
    {
        Number = number;
        Course = course;
        IsValid = true;
    }

    // Метод структуры
    public string Describe() =>
        $"билет {Number}, {Course} курс, " + (IsValid ? "действителен" :
"недействителен");
}
```

Auditorium.cs

```
struct Auditorium
{
    public string Number;
    public int Seats;

    // Конструктор без параметров (C# 10+)
    public Auditorium()
    {
        Number = "не назначена";
        Seats = 0;
    }

    // Конструктор с параметрами
    public Auditorium(string number, int seats)
    {
        Number = number;
        Seats = seats;
    }

    public Auditorium(string number) : this(number, 10){}

    // Цепочка конструкторов через `: this(...)` работает так же, как у
классов.

    // Метод структуры
    public string Describe()
    {
        return $"Аудитория {Number} на {Seats} мест";
    }

    public override string ToString()
    {
        return Describe();
    }
}
```

Student.cs

```
namespace MySpace;
class Student
{

    private const string UNKNOWN = "Unknown" ;
```

```
    // Поля (состояние экземпляра)
    private string? Name;
    private int Age;

    // Конструктор без параметров
    public Student()
    {
        Name = UNKNOWN;
        Age = 0;
    }

    // Конструктор с параметрами
    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }

    // Методы (доступные действия экземпляра)
    public string? GetName(){
        return string.IsNullOrEmpty(Name) ? UNKNOWN : Name;
    }

    public int GetAge(){
        return Age;
    }

    public void SetName(string Name){
        this.Name = Name;
    }

    public void SetAge(int Age){
        this.Age = Age;
    }

    public override string ToString()
    {
        return $"Студент {Name} возраст {Age} лет";
    }
}
```

PresenceRecord.cs

```
using MySpace;

namespace MySubSpace
{
    struct AnyStruct
    {
        public string name;
    }
}

struct PresenceRecord
{
    public StudentCard studentCard;
    public Auditorium auditorium;
    public DateTime dateTime;

    public PresenceRecord(StudentCard card, Auditorium auditorium)
    {
        studentCard = card;
        this.auditorium = auditorium;
        dateTime = DateTime.Now;
    }

    public override string ToString()
    {
        return $"{dateTime} в аудитории {auditorium} был найден
{studentCard.Describe()}";
    }
}
```